JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.zip

Class Deflater

java.lang.Object
    java.util.zip.Deflater

```
public class Deflater
extends Object
```

This class provides support for general purpose compression using the popular ZLIB compression library. The ZLIB compression library was initially developed as part of the PNG graphics standard and is not protected by patents. It is fully described in the specifications at the java.util.zip package description.

The following code fragment demonstrates a trivial compression and decompression of a string using `Deflater` and `Inflater`.

```
try {
    // Encode a String into bytes
    String inputString = "blahblahblah";
    byte[] input = inputString.getBytes("UTF-8");

    // Compress the bytes
    byte[] output = new byte[100];
    Deflater compresser = new Deflater();
    compresser.setInput(input);
    compresser.finish();
    int compressedDataLength = compresser.deflate(output);
    compresser.end();

    // Decompress the bytes
    Inflater decompresser = new Inflater();
    decompresser.setInput(output, 0, compressedDataLength);
    byte[] result = new byte[100];
    int resultLength = decompresser.inflate(result);
```

```
        decompresser.end();

        // Decode the bytes into a String
        String outputString = new String(result, 0, resultLength, "UTF-8");
    } catch(java.io.UnsupportedEncodingException ex) {
        // handle
    } catch (java.util.zip.DataFormatException ex) {
        // handle
    }
```

See Also:

Inflater

Field Summary

Fields

Modifier and Type	Field	Description
static int	**BEST_COMPRESSION**	Compression level for best compression.
static int	**BEST_SPEED**	Compression level for fastest compression.
static int	**DEFAULT_COMPRESSION**	Default compression level.
static int	**DEFAULT_STRATEGY**	Default compression strategy.
static int	**DEFLATED**	Compression method for the deflate algorithm (the only one currently supported).
static int	**FILTERED**	Compression strategy best used for data consisting mostly of small values with a somewhat random distribution.
static int	**FULL_FLUSH**	Compression flush mode used to flush out all pending output and reset the deflater.
static int	**HUFFMAN_ONLY**	Compression strategy for Huffman coding only.
static int	**NO_COMPRESSION**	Compression level for no compression.

static int	NO_FLUSH	Compression flush mode used to achieve best compression result.
static int	SYNC_FLUSH	Compression flush mode used to flush out all pending output; may degrade compression for some compression algorithms.

Constructor Summary

Constructors

Constructor	Description
Deflater()	Creates a new compressor with the default compression level.
Deflater(int level)	Creates a new compressor using the specified compression level.
Deflater(int level, boolean nowrap)	Creates a new compressor using the specified compression level.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
int	deflate(byte[] b)	Compresses the input data and fills specified buffer with compressed data.
int	deflate(byte[] b, int off, int len)	Compresses the input data and fills specified buffer with compressed data.
int	deflate(byte[] b, int off, int len, int flush)	Compresses the input data and fills the specified buffer with compressed data.
void	end()	

		Closes the compressor and discards any unprocessed input.
protected void	**finalize**()	Closes the compressor when garbage is collected.
void	**finish**()	When called, indicates that compression should end with the current contents of the input buffer.
boolean	**finished**()	Returns true if the end of the compressed data output stream has been reached.
int	**getAdler**()	Returns the ADLER-32 value of the uncompressed data.
long	**getBytesRead**()	Returns the total number of uncompressed bytes input so far.
long	**getBytesWritten**()	Returns the total number of compressed bytes output so far.
int	**getTotalIn**()	Returns the total number of uncompressed bytes input so far.
int	**getTotalOut**()	Returns the total number of compressed bytes output so far.
boolean	**needsInput**()	Returns true if the input data buffer is empty and setInput() should be called in order to provide more input.

void	**reset**()	Resets deflater so that a new set of input data can be processed.
void	**setDictionary**(byte[] b)	Sets preset dictionary for compression.
void	**setDictionary**(byte[] b, int off, int len)	Sets preset dictionary for compression.
void	**setInput**(byte[] b)	Sets input data for compression.
void	**setInput**(byte[] b, int off, int len)	Sets input data for compression.
void	**setLevel**(int level)	Sets the compression level to the specified value.
void	**setStrategy**(int strategy)	Sets the compression strategy to the specified value.

Methods inherited from class java.lang.Object

clone, equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

DEFLATED

```
public static final int DEFLATED
```

Compression method for the deflate algorithm (the only one currently supported).

See Also:

Constant Field Values

NO_COMPRESSION

```
public static final int NO_COMPRESSION
```

Compression level for no compression.

See Also:

Constant Field Values

BEST_SPEED

```
public static final int BEST_SPEED
```

Compression level for fastest compression.

See Also:

Constant Field Values

BEST_COMPRESSION

```
public static final int BEST_COMPRESSION
```

Compression level for best compression.

See Also:

Constant Field Values

DEFAULT_COMPRESSION

```
public static final int DEFAULT_COMPRESSION
```

Default compression level.

See Also:

Constant Field Values

FILTERED

```
public static final int FILTERED
```

Compression strategy best used for data consisting mostly of small values with a somewhat random distribution. Forces more Huffman coding and less string matching.

See Also:

Constant Field Values

HUFFMAN_ONLY

```
public static final int HUFFMAN_ONLY
```

Compression strategy for Huffman coding only.

See Also:

Constant Field Values

DEFAULT_STRATEGY

```
public static final int DEFAULT_STRATEGY
```

Default compression strategy.

See Also:

Constant Field Values

NO_FLUSH

```
public static final int NO_FLUSH
```

Compression flush mode used to achieve best compression result.

Since:

1.7

See Also:

deflate(byte[], int, int, int), Constant Field Values

SYNC_FLUSH

```
public static final int SYNC_FLUSH
```

Compression flush mode used to flush out all pending output; may degrade compression for some compression algorithms.

Since:

1.7

See Also:

deflate(byte[], int, int, int), Constant Field Values

FULL_FLUSH

```
public static final int FULL_FLUSH
```

Compression flush mode used to flush out all pending output and reset the deflater. Using this mode too often can seriously degrade compression.

Since:

1.7

See Also:

`deflate(byte[], int, int, int)`, `Constant Field Values`

Constructor Detail

Deflater

```
public Deflater(int level,
                boolean nowrap)
```

Creates a new compressor using the specified compression level. If 'nowrap' is true then the ZLIB header and checksum fields will not be used in order to support the compression format used in both GZIP and PKZIP.

Parameters:

`level - the compression level (0-9)`

`nowrap - if true then use GZIP compatible compression`

Deflater

```
public Deflater(int level)
```

Creates a new compressor using the specified compression level. Compressed data will be generated in ZLIB format.

Parameters:

`level - the compression level (0-9)`

Deflater

```
public Deflater()
```

Creates a new compressor with the default compression level. Compressed data will be generated in ZLIB format.

Method Detail

setInput

```
public void setInput(byte[] b,
                     int off,
                     int len)
```

Sets input data for compression. This should be called whenever needsInput() returns true indicating that more input data is required.

Parameters:

`b - the input data bytes`

`off - the start offset of the data`

`len - the length of the data`

See Also:

`needsInput()`

setInput

```
public void setInput(byte[] b)
```

Sets input data for compression. This should be called whenever needsInput() returns true indicating that more input data is required.

Parameters:

`b - the input data bytes`

See Also:

`needsInput()`

setDictionary

```
public void setDictionary(byte[] b,
                          int off,
                          int len)
```

Sets preset dictionary for compression. A preset dictionary is used when the history buffer can be predetermined. When the data is later uncompressed

with Inflater.inflate(), Inflater.getAdler() can be called in order to get the Adler-32 value of the dictionary required for decompression.

Parameters:

`b - the dictionary data bytes`

`off - the start offset of the data`

`len - the length of the data`

See Also:

`Inflater.inflate(byte[], int, int)`, `Inflater.getAdler()`

setDictionary

```
public void setDictionary(byte[] b)
```

Sets preset dictionary for compression. A preset dictionary is used when the history buffer can be predetermined. When the data is later uncompressed with Inflater.inflate(), Inflater.getAdler() can be called in order to get the Adler-32 value of the dictionary required for decompression.

Parameters:

`b - the dictionary data bytes`

See Also:

`Inflater.inflate(byte[], int, int)`, `Inflater.getAdler()`

setStrategy

```
public void setStrategy(int strategy)
```

Sets the compression strategy to the specified value.

If the compression strategy is changed, the next invocation of `deflate` will compress the input available so far with the old strategy (and may be flushed); the new strategy will take effect only after that invocation.

Parameters:

`strategy - the new compression strategy`

Throws:

`IllegalArgumentException - if the compression strategy is invalid`

setLevel

```
public void setLevel(int level)
```

Sets the compression level to the specified value.

If the compression level is changed, the next invocation of `deflate` will compress the input available so far with the old level (and may be flushed); the new level will take effect only after that invocation.

Parameters:

`level - the new compression level (0-9)`

Throws:

`IllegalArgumentException - if the compression level is invalid`

needsInput

```
public boolean needsInput()
```

Returns true if the input data buffer is empty and setInput() should be called in order to provide more input.

Returns:

`true if the input data buffer is empty and setInput() should be called in order to provide more input`

finish

```
public void finish()
```

When called, indicates that compression should end with the current contents of the input buffer.

finished

```
public boolean finished()
```

Returns true if the end of the compressed data output stream has been reached.

Returns:

`true if the end of the compressed data output stream has been reached`

deflate

```
public int deflate(byte[] b,
                   int off,
                   int len)
```

Compresses the input data and fills specified buffer with compressed data. Returns actual number of bytes of compressed data. A return value of 0 indicates that `needsInput` should be called in order to determine if more input data is required.

This method uses `NO_FLUSH` as its compression flush mode. An invocation of this method of the form `deflater.deflate(b, off, len)` yields the same result as the invocation of `deflater.deflate(b, off, len, Deflater.NO_FLUSH)`.

Parameters:

`b - the buffer for the compressed data`

`off - the start offset of the data`

`len - the maximum number of bytes of compressed data`

Returns:

`the actual number of bytes of compressed data written to the output buffer`

deflate

```
public int deflate(byte[] b)
```

Compresses the input data and fills specified buffer with compressed data. Returns actual number of bytes of compressed data. A return value of 0 indicates that `needsInput` should be called in order to determine if more input data is required.

This method uses `NO_FLUSH` as its compression flush mode. An invocation of this method of the form `deflater.deflate(b)` yields the same result as the invocation of `deflater.deflate(b, 0, b.length, Deflater.NO_FLUSH)`.

Parameters:

`b - the buffer for the compressed data`

Returns:

`the actual number of bytes of compressed data written to the output buffer`

deflate

```
public int deflate(byte[] b,
                   int off,
                   int len,
                   int flush)
```

Compresses the input data and fills the specified buffer with compressed data. Returns actual number of bytes of data compressed.

Compression flush mode is one of the following three modes:

- `NO_FLUSH`: allows the deflater to decide how much data to accumulate, before producing output, in order to achieve the best compression (should be used in normal use scenario). A return value of 0 in this flush mode indicates that `needsInput()` should be called in order to determine if more input data is required.
- `SYNC_FLUSH`: all pending output in the deflater is flushed, to the specified output buffer, so that an inflater that works on compressed data can get all input data available so far (In particular the `needsInput()` returns `true` after this invocation if enough output space is provided). Flushing with `SYNC_FLUSH` may degrade compression for some compression algorithms and so it should be used only when necessary.
- `FULL_FLUSH`: all pending output is flushed out as with `SYNC_FLUSH`. The compression state is reset so that the inflater that works on the compressed output data can restart from this point if previous compressed data has been damaged or if random access is desired. Using `FULL_FLUSH` too often can seriously degrade compression.

In the case of `FULL_FLUSH` or `SYNC_FLUSH`, if the return value is `len`, the space available in output buffer `b`, this method should be invoked again with the same `flush` parameter and more output space.

Parameters:

`b - the buffer for the compressed data`

`off - the start offset of the data`

`len - the maximum number of bytes of compressed data`

`flush - the compression flush mode`

Returns:

`the actual number of bytes of compressed data written to the output buffer`

Throws:

`IllegalArgumentException - if the flush mode is invalid`

Since:

`1.7`

getAdler

```
public int getAdler()
```

Returns the ADLER-32 value of the uncompressed data.

Returns:

the ADLER-32 value of the uncompressed data

getTotalIn

```
public int getTotalIn()
```

Returns the total number of uncompressed bytes input so far.

Since the number of bytes may be greater than Integer.MAX_VALUE, the `getBytesRead()` method is now the preferred means of obtaining this information.

Returns:

the total number of uncompressed bytes input so far

getBytesRead

```
public long getBytesRead()
```

Returns the total number of uncompressed bytes input so far.

Returns:

the total (non-negative) number of uncompressed bytes input so far

Since:

1.5

getTotalOut

```
public int getTotalOut()
```

Returns the total number of compressed bytes output so far.

Since the number of bytes may be greater than Integer.MAX_VALUE, the `getBytesWritten()` method is now the preferred means of obtaining this information.

Returns:

the total number of compressed bytes output so far

getBytesWritten

```
public long getBytesWritten()
```

Returns the total number of compressed bytes output so far.

Returns:

the total (non-negative) number of compressed bytes output so far

Since:

1.5

reset

```
public void reset()
```

Resets deflater so that a new set of input data can be processed. Keeps current compression level and strategy settings.

end

```
public void end()
```

Closes the compressor and discards any unprocessed input. This method should be called when the compressor is no longer being used, but will also be called automatically by the finalize() method. Once this method is called, the behavior of the Deflater object is undefined.

finalize

```
protected void finalize()
```

Closes the compressor when garbage is collected.

Overrides:

finalize in class Object

See Also:

```
WeakReference, PhantomReference
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.